

# Chapter 7

## *Bean Validation & Global Exception Handling*



Rejecting bad data before it reaches the database, and turning validation failures into clean, predictable JSON responses.

---

## 1. What is Validation?

---

Validation is the process of checking whether the data sent by the client satisfies the required rules before it is processed or stored in the database. It ensures that only valid data enters the application.

- Food name should not be empty
- Quantity should be at least 1
- Expiry time should be in the future

## 2. Why Do We Need Validation?

---

- Prevent invalid data from entering the database
- Improve application security
- Reduce unnecessary validation logic in the service layer
- Return meaningful error messages to the client
- Improve user experience
- Keep business logic clean

## 3. Validation Dependency

---

```
<dependency>
  <groupId>org.springframework.boot</groupId>
  <artifactId>spring-boot-starter-validation</artifactId>
</dependency>
```

This enables Bean Validation in Spring Boot and provides validation annotations.

## 4. Validation Annotations

| ANNOTATION | APPLIES TO              | CHECKS                                                  |
|------------|-------------------------|---------------------------------------------------------|
| @NotBlank  | String values           | Not null, not empty, not only whitespace                |
| @Min       | Numeric values          | Value is greater than or equal to the specified minimum |
| @Future    | Date / Timestamp values | Value is in the future                                  |

```
@NotBlank(message = "Food name is required")
private String foodName;

@Min(value = 1, message = "Quantity must be at least one")
private int quantity;

@Future(message = "Expiry time must be in future")
private Timestamp expiryTime;
```

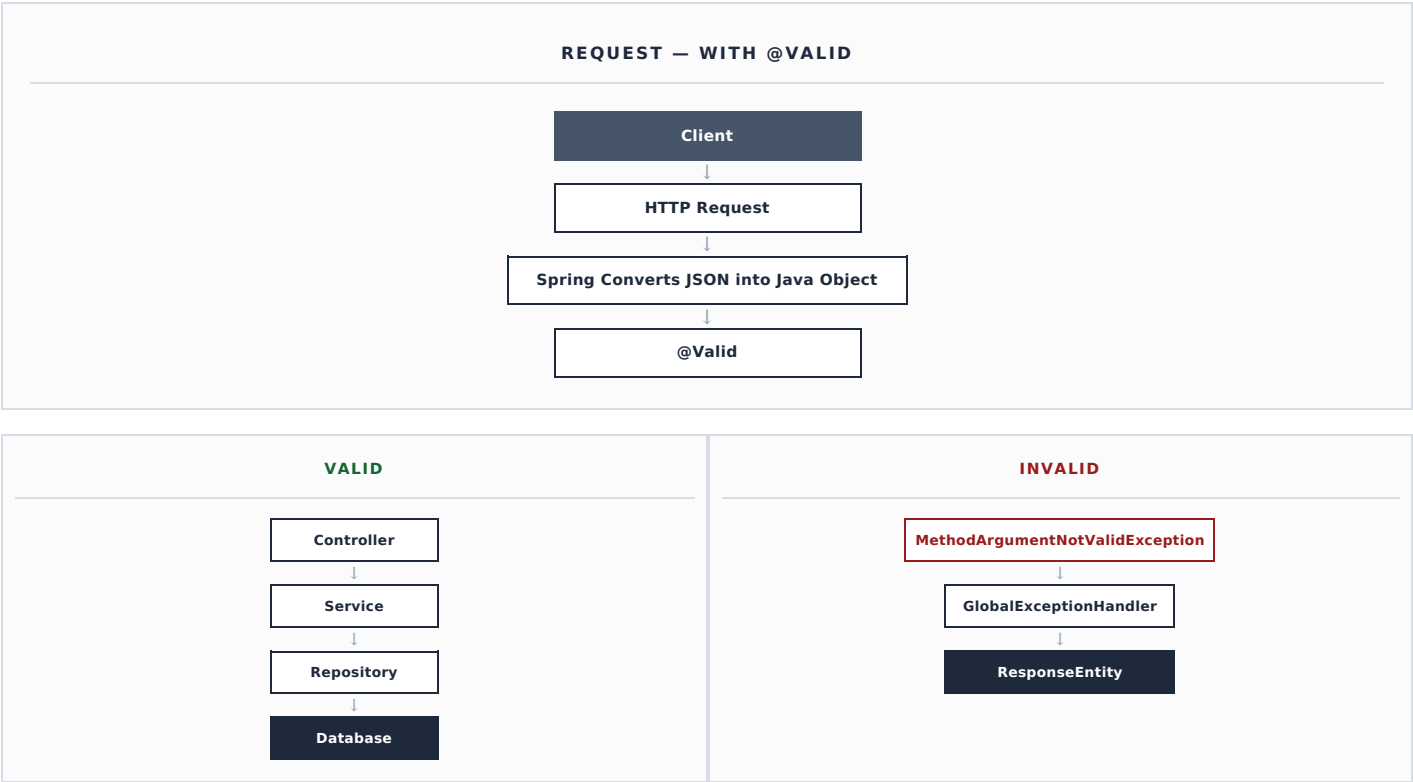
## 5. @Valid Annotation

```
@PostMapping
public ResponseEntity<FoodDonation> createDonation(
    @Valid @RequestBody FoodDonation foodDonation){

    return ResponseEntity
        .status(HttpStatus.CREATED)
        .body(foodDonationService.createDonation(foodDonation));
}
```

- Tells Spring to validate the request body
- Validation happens before the controller method executes
- If validation fails, Spring throws `MethodArgumentNotValidException`
- The controller method is not executed if validation fails

## 6. Validation Flow



## 7. What Happens if @Valid is Missing?

Without `@Valid`, Spring does not validate the request. The request reaches the controller, the service method executes, and `Repository.save()` is called. Hibernate then performs validation before inserting or updating — and if it fails, Hibernate throws a `ConstraintViolationException` instead.



## 8. Global Exception Handling — @ControllerAdvice

---

`@ControllerAdvice` handles exceptions globally, eliminating duplicate try-catch blocks, making controllers cleaner, and returning consistent error responses:

```
@ControllerAdvice
public class GlobalExceptionHandler {

}
```

## 9. @ExceptionHandler

---

Specifies which exception a method should handle:

```
@ExceptionHandler(IllegalArgumentException.class)
```

Whenever an `IllegalArgumentException` occurs, Spring automatically calls the corresponding method.

## 10. Handling IllegalArgumentException

---

```
@ExceptionHandler(IllegalArgumentException.class)
public ResponseEntity<String> illegalArgumentExceptionHandler(
    IllegalArgumentException ex){

    return ResponseEntity
        .badRequest()
        .body(ex.getMessage());
}
```

Example response:

```
400 Bad Request

Donation doesn't exist
```

## 11-14. MethodArgumentNotValidException, BindingResult & FieldError

Whenever `@Valid` detects invalid input, Spring throws `MethodArgumentNotValidException` . Instead of letting Spring return a huge default response, we catch it ourselves inside `GlobalExceptionHandler` .

The exception contains a `BindingResult` , which stores all validation errors:

```
ex.getBindingResult()
```

`BindingResult` contains a list of `FieldError` objects. Each `FieldError` stores:

| PROPERTY   | EXAMPLE               |
|------------|-----------------------|
| Field Name | <code>foodName</code> |
| Message    | Food name is required |

### 14. getFieldErrors()

Returns all validation errors:

```
List<FieldError> fieldErrors =  
    ex.getBindingResult().getFieldErrors();
```

### 15. Why Do We Use Map<String, String>?

We want to return a clean JSON response — the map key is the field name, and the value is the validation message:

| FIELD         | MESSAGE                       |
|---------------|-------------------------------|
| foodName      | Food name is required         |
| quantity      | Quantity must be at least one |
| pickupAddress | Pickup address is required    |
| expiryTime    | Expiry time must be in future |

## 16. Complete Validation Exception Handler

```
@ExceptionHandler(MethodArgumentNotValidException.class)
public ResponseEntity<Map<String,String>> methodArgumentNotValidExceptionHandler(
    MethodArgumentNotValidException ex){

    Map<String,String> errors = new HashMap<>();

    for(FieldError error : ex.getBindingResult().getFieldErrors()){

        errors.put(
            error.getField(),
            error.getDefaultMessage()
        );
    }

    return ResponseEntity
        .badRequest()
        .body(errors);
}
```

## 17. How the for Loop Works

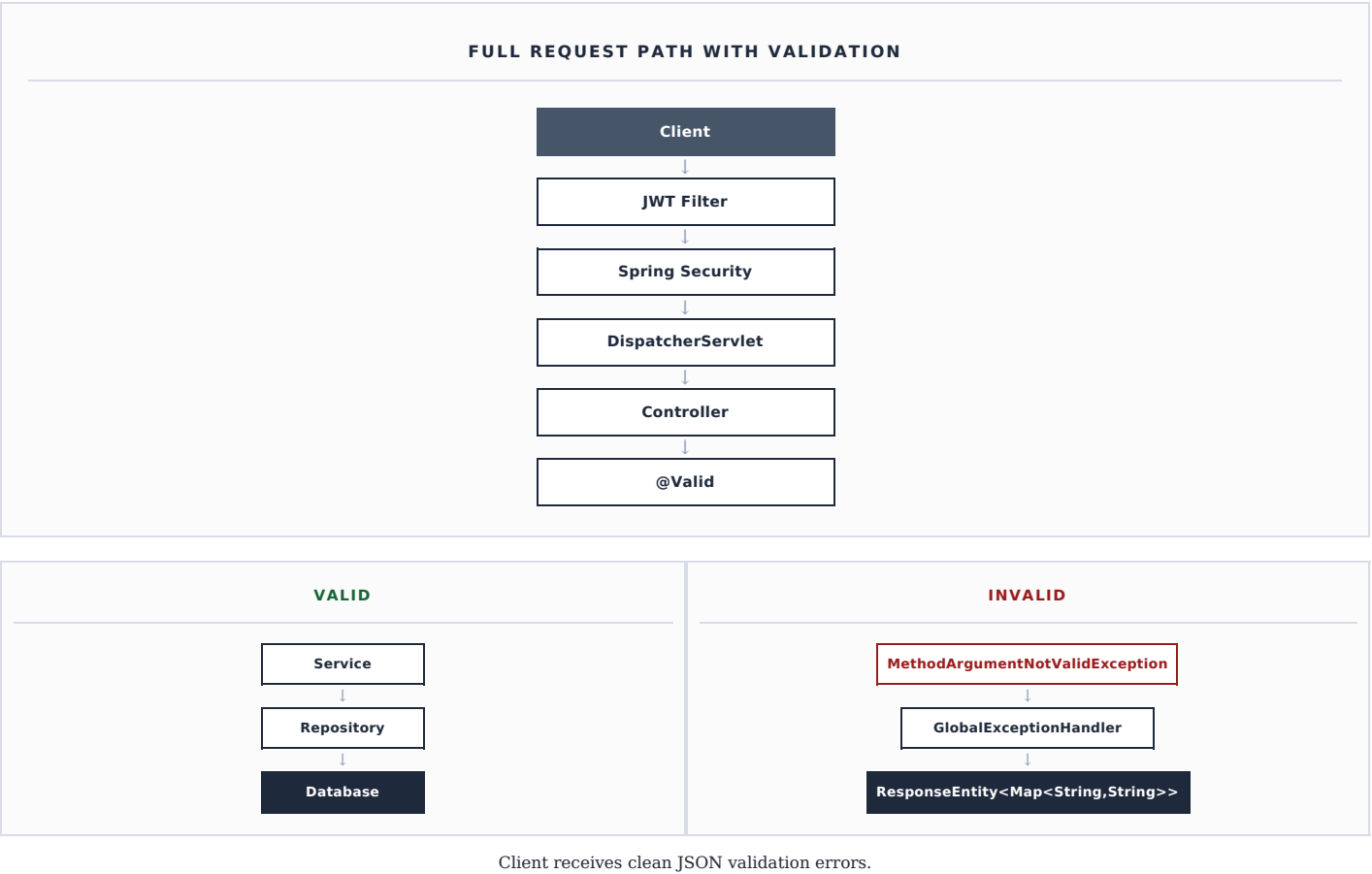
`ex.getBindingResult().getFieldErrors()` returns a `List<FieldError>` . The enhanced for loop takes one `FieldError` object at a time:

| ITERATION | FIELD         | MESSAGE                       |
|-----------|---------------|-------------------------------|
| 1         | foodName      | Food name is required         |
| 2         | quantity      | Quantity must be at least one |
| 3         | pickupAddress | Pickup address is required    |
| 4         | expiryTime    | Expiry time must be in future |

Final map:

```
{
  "foodName": "Food name is required",
  "quantity": "Quantity must be at least one",
  "pickupAddress": "Pickup address is required",
  "expiryTime": "Expiry time must be in future"
}
```

## 18. Final Request Flow





## Interview Questions

---

### Q. Why do we use @Valid?

It tells Spring to validate the request body before the controller method executes.

### Q. Why do we use @ControllerAdvice?

It provides global exception handling so that we don't have to write try-catch blocks in every controller.

### Q. Why do we use @ExceptionHandler?

It tells Spring which exception a method should handle.

### Q. Why do we use BindingResult?

BindingResult stores all validation errors generated during request validation.

### Q. Why do we use FieldError?

FieldError contains the name of the field that failed validation and its corresponding validation message.

### Q. Why do we return Map<String, String>?

Because it creates a clean, lightweight JSON response that is easy for frontend applications like React to display.

## CHAPTER SUMMARY

Bean Validation lets Spring reject malformed requests before they ever reach the service or repository layers. Annotations such as `@NotBlank`, `@Min`, and `@Future` declare the rules directly on the entity, and `@Valid` triggers those checks automatically in the controller.

*When validation fails, a global `@ControllerAdvice` handler catches the resulting `MethodArgumentNotValidException`, walks its `BindingResult`, and returns a clean `Map<String, String>` of field-level errors — giving the client a predictable, easy-to-render JSON response instead of a raw stack trace.*